

COVER_TRANS

Overview

cover_trans collects transitional coverage on the multi-bit data input. Users may define any number of transitions and the minimum & maximum values for each of the transition counters. When ALL counters reach their minimum limits, the **min_limit** access event toggles. When ONE of the counters reaches its maximum limit, the **max_limit** access event toggles. All coverage information can be accessed dynamically in order to tune test the generation process and define additional constraints and length of the simulation run. One of the possible applications is getting transitional coverage from the FSM.

Syntax

```
cover_trans #(bit_width,cover_num, ssize) InstName (clk, rst_n, data);
```

Parameters		
Bit_width	Bit width of the data input	
cover_num	Maximum number of coverage items allowed. Default: 0	
Ssize	String size for the name of cover item. Default: 20	
Interface		
clk	Sampling clock for data	
rst_n	Active-low reset	
data	Input data signal, for coverage collection. Bit width of this signal has to be defined using “bit_width” parameter.	
Access Tasks		
Name	Arguments	Description
reset	No	Resets coverage counters and maximum & minimum limits
add	cover_name from_state to_state min_limit max_limit	- name of the coverage Item (with length <= ssize) - transition’s starting state - transition’s ending state - lower limit for the counter. Default: 0 - Upper limit for the counter. Default: 32’hFFFF_FFFF Use “add” access task to add functional Items to the vc_cover coverage group. Example: <InstName>.add(("S1->S2", S1, S2, 5, 8); – add coverage item for transition S1->S2, lower limit eq. to 5and upper limit equals to 8
change	cover_name from_state to_state count min_limit max_limit	- name of the coverage Item (with length <= ssize) - transition’s starting state - transition’s ending state - counter number for the coverage item - lower limit for the counter. Default: 0 - Upper limit for the counter. Default: 32’hFFFF_FFFF Use “change” access task to edit functional Item data. Example: <InstName>.edit(((“S1->S2”, S1, S2, 0, 5, 10) – resets counter and change max. limit from 8 to 10 for the coverage transition “S1->s2”
report	No	Generates coverage report in tabular format. See below an example of a report.
Access Events		
covered	“Fires” when all vc_cover counters exceed their lower limits. Can be monitored externally to define the length of the test case execution based on the accumulated coverage information.	

exceeded	"Fires" when any of the counters exceeds a predefined maximum limit. The maximum limits can be used as additional test-specific constraints. When it fires, "Exceeded_name" access register provides the name of the cover item exceeding the maximum limit.
Access Variables	
total_cov	Provides a constantly updated value of the total coverage (in %). After a "covered" event, it has to be equal to 100 %.
exceeded_name	The name of the coverage item with the counter exceeding a predefined maximum value.
Control Variables	
Display_status	Set it to "1" to get more informational messages. Default: 0

Verilog example

The example below shows how coverage information controls the simulation run. By monitoring a "covered" event, the test case is running until a defined coverage is accumulated. By monitoring an "exceeded" event, the test case exits upon the first violation.

```

module test;

reg clk; initial clk = 0;
reg [2:0] data;

parameter INIT = 0;
parameter S1 = 1;
parameter S2 = 2;
parameter S3 = 3;
parameter S4 = 4;
parameter S5 = 5;
parameter S6 = 6;
parameter S7 = 7;

always #10 clk <= ~clk;

always @(posedge clk)
    data <= ($random)%7;

//-----
cover_trans #(3,8) cov (clk, rst_n, data);

initial begin
    cov.display_status = 0;
    //      name      from  to  min max
    //      -----
    cov.add("INIT->S1", INIT, S1, 5, 8);
    cov.add("S1->S2", S1, S2, 5, 8);
    cov.add("S1->S5", S1, S5, 5, 8);
    cov.add("S5->S3", S5, S3, 6, 8);
    cov.add("S6->S3", S6, S3, 5, 8); // exit on coverage collection
//cov.add("S6->S3", S6, S3, 3, 6); // exit on reaching max limit
end
//-----

initial forever begin
    #1000 cov.report;
end

// Exit on first maximum violation

```

```

always @(cov.exceeded) begin
    $display ("[%0t] TC INFO: MAXIMUM reached for '%s'",
        $time, cov.exceeded_name);
    $finish;
end

// Run till all functional coverage is collected; then exit
always @(cov.covered) begin
    $display("[%0t] TC INFO: Stimulus coverage reached, test may exit", $time);
    cov.report;
    $finish;
end

//If functional coverage is not collected, use timebomb
initial begin
    $dumpvars;
    #10000 $finish;
end

endmodule

```

Coverage Report Example

Transition Coverage Report <test.cov.report>				
Cover name	Count	Min_limit	Max_limit	Coverage
INIT->S1	4	5	8	80 percent
S1->S2	5	5	8	100 percent
S1->S5	5	5	8	100 percent
S5->S3	6	6	8	100 percent
S6->S3	5	5	8	100 percent
Total Coverage:	96 percent			